

03_Polynomial_Time_Computable_Problems

Saturday, April 11, 2026 11:45 AM

- ▶ A **complexity class** is a set of *tasks* which can be computed within some prescribed resource bounds.
 - ▶ It is *not* a set of TMs, although it is defined based on TMs.
 - ▶ Typically, the task we are interested at are decision problems, or equivalently languages (i.e. subsets of $\{0, 1\}^*$).

A **complexity class** is a collection of computational tasks that can be solved using a specific amount of resources, such as time or memory. The class $DTIME(T(n))$ specifically measures the **deterministic time** required to solve a problem.

To understand the definition—"Let $T : \mathbb{N} \rightarrow \mathbb{N}$. A language L is in the class $DTIME(T(n))$ iff there is a TM deciding L and running in time $n \mapsto c \cdot T(n)$ for some constant c "—we can break it down into its core components:

- $T : \mathbb{N} \rightarrow \mathbb{N}$: This is a function that maps a natural number (the input size) to another natural number (the maximum number of steps allowed). It acts as a "**speed limit**" for the computation. "This input of length N_1 approximately require N_2 steps"
- **Language L** : In this context, a language is a **set of binary strings** that represent the "yes" instances of a decision problem.
- **IFF (If and Only If)**: This indicates a **strict equivalence**; a language belongs to this class if it meets the criteria, and if it is in the class, it must have met those criteria.
- **Turing Machine (TM)**: This is the **standard abstract model** of a computer used to measure efficiency.
- **Deciding L** : A TM "decides" a language if it **always halts** and gives a correct "yes" or "no" answer for every possible input string.
- **Time $n \mapsto c \cdot T(n)$** : Here, **n represents the length of the input string**. The running time is the number of basic instructions the machine executes.
- **Constant c** : This is a fixed positive number. Including a constant c makes the definition **robust** because it ensures the class doesn't change just because you use a slightly different (but equivalent) machine model that might be a bit faster or slower by a fixed factor.

Practical Example: The Palindrome Problem

Consider the task of determining if a binary string is a **palindrome** (it reads the same forwards and backwards).

1. **The Language (L)**: The set of all strings like 0110, 101, or 1111.
2. **The Time Bound ($T(n)$)**: If we have an algorithm that checks the string by comparing the first and last characters, then moving inward, it might take a number of steps proportional to the length of the string, n .
3. **The Constant (c)**: On one type of Turing Machine, this might take exactly $3n$ steps. In this case, $T(n) = n$ and the constant $c = 3$.
4. **The Classification**: Because there exists a TM that can decide whether any string is a palindrome within $3n$ steps, we can say that the **Language of Palindromes is in the class $DTIME(n)$** .

The Class P

The **Complexity Class P** (standing for **Polynomial Time**) is the collection of all computational tasks

that are considered **efficiently solvable** or **tractable**.

To understand the formal definition provided in the sources, we can break it down into three specific layers:

1. The Mathematical Definition: $P = \bigcup_{c \geq 1} DTIME(n^c)$

This formula tells us that the class P is not just one group of problems, but a **union** of many different classes.

- **$DTIME(n^c)$** : This represents a specific "speed category". For example, $DTIME(n^2)$ includes problems that can be solved in a number of steps proportional to the square of the input size.
- **The Union Symbol ($\bigcup_{c \geq 1}$)**: This means we are gathering every language that fits into *any* polynomial category, whether it is n^1, n^2, n^3 , or even n^{1000} . If there exists **any constant** $c \geq 1$ such that a machine can solve the problem in $O(n^c)$ steps, the problem is officially in P .

2. Decidability by a Turing Machine (TM)

A language L is in P only if there is a **Deterministic Turing Machine** that "decides" it.

- **Determinism**: This means the machine follows a fixed, predictable set of instructions where the next step is always clearly determined.
- **Deciding**: To "decide" a language, the TM must **always halt** and provide a correct "yes" or "no" answer (0 or 1) for every possible input string. If a machine could run forever on some inputs, it would not be a "decider".

3. Working in Polynomial Time

The sources specify that the machine must run in time P , where P is **any polynomial**.

- **Input Size (n)**: The "time" is measured by the number of basic instructions the machine performs relative to the length of the input string n .
- **Robustness**: Using polynomials is a "robust" choice because most realistic computer models can simulate each other with only a polynomial difference in speed. This means that a problem in P on a Turing Machine will also be in P on a modern laptop.
- **Practicality**: While theoretically n^{100} is a polynomial, in practice, most problems in P have much smaller exponents, like n^2 (quadratic) or n^3 (cubic).

Other characteristics of Polynomial Class

1. Polynomial Bounding

"Indeed, for any polynomial P there are $c, d > 0$ such that $P(n) \leq c \cdot n^d$ for sufficiently large n ." This statement explains why the formal definition of P —the union of all $DTIME(n^c)$ —is sufficient to capture every possible polynomial.

- **The Logic**: A polynomial is essentially "controlled" by its highest-degree term. For any polynomial (e.g., $5n^3 + 20n^2 + 100$), you can always find a constant (c) and a power (d) such that $c \cdot n^d$ eventually becomes and stays larger than the original polynomial for all inputs after a certain point N .
- **Asymptotic Notation**: This is the foundation of **Big O notation**, where we focus on the growth rate rather than the specific details of the function.

2. Arbitrarily Large Constants

"Please observe that c and d can be arbitrarily large, so a TM deciding L and working in time $10^{20} \cdot n^{10^{30}}$ is a witness of L being in P ." This highlights the **theoretical breadth** of the class P .

- **Theoretical vs. Practical**: Mathematically, a "constant" is any fixed number that does not change regardless of how large the input n becomes. Even if the constant (10^{20}) or the exponent (10^{30}) is massive, it still fits the mathematical definition of a polynomial.
- **A "Witness"**: In complexity theory, a "witness" is evidence that a language belongs to a certain class. If you can prove a Turing Machine exists that solves the problem in $10^{20} \cdot n^{10^{30}}$

steps, you have successfully proven that the problem is in **P**.

3. P as the Class of Efficient Languages

"**P** is generally considered as the class of efficiently decidable languages." While n^{1000} is not "efficient" in the real world, **P** is used as the gold standard for **tractability** for several reasons:

- **Robustness:** The class **P** is independent of the specific computer model used. According to the **Strong Church-Turing Thesis**, any physically realistic computer can simulate a Turing Machine with only a **polynomial overhead**. This means a problem in **P** will stay in **P** whether you run it on a Turing Machine or a modern supercomputer.
- **Small Exponents in Practice:** Although the definition allows for massive exponents, the sources note that for most problems of actual interest, researchers find algorithms with much smaller bounds, such as **quadratic** (n^2) or **cubic** (n^3).
- **Mathematical Closure:** **P** has "nice closure properties," meaning you can combine multiple polynomial-time algorithms (using loops or subroutines) and the resulting total time will still be a polynomial.

► The Strong Church-Turing Thesis

- Every physically realizable computer can be simulated by a TM with a *polynomial* overhead in time (n steps on the computer requires n^c on TMs, where c only depends on the computer), and viceversa.
- The class **P** would be *the same* if defined based on other realistic models of computation.

Some Criticisms on P

Worst-Case Complexity is Not Always Realistic

The definition of **P** is inherently based on **worst-case complexity**. For a problem to be in **P**, there must exist a Turing Machine and a polynomial such that **every single possible input** is solved within that time limit.

- **The Criticism:** In the real world, many algorithms are considered highly "efficient" because they work very fast on the types of inputs that actually occur in practice. However, if a problem has even one "pathological" or extremely rare set of inputs that takes exponential time, that problem is technically excluded from **P**, even if it runs in seconds for 99.9% of users.
- **Example (from the context of SAT solvers):** While the general SAT problem is NP-complete (and not known to be in **P**), modern **SAT-solvers** work extremely well on a large class of practical formulas, even though they do not run in polynomial time for *every* possible formula.

Proposed Solutions:

- **Average-case Complexity:** Instead of looking at the hardest possible input, this approach measures the expected time an algorithm takes on an "average" input drawn from a specific distribution.
- **Approximation Algorithms:** If finding the perfect "yes/no" or optimal answer is too slow, we can use algorithms that find an answer that is "**good enough**" (e.g., within 1% of the best possible result) in polynomial time.

Alternative Computational Models

The class **P** is traditionally defined using **Deterministic Turing Machines**. However, our understanding of what is "feasible" has expanded beyond simple deterministic steps.

- **The Criticism:** Some computational models use resources that a standard TM does not. For example, some problems might be solved much faster using **randomness** (flipping a coin to decide the next step) or **quantum mechanics**. If a problem can be solved efficiently by a quantum computer but not a classical one, then the traditional class P fails to capture the full scope of "efficient" computation.

Proposed Redefinitions:

To address these alternative ways of computing, complexity theory has introduced new classes that serve as the "P" of those models:

- **BPP (Bounded-error Probabilistic Polynomial time):** This class defines what is efficiently solvable if the computer is allowed to use **randomized computation** and has a small, controlled chance of making an error.
- **BQP (Bounded-error Quantum Polynomial time):** This is the equivalent of P for **quantum computers**. It includes problems that can be solved in polynomial time using quantum bits (qubits) and quantum gates.

In summary, while P remains a robust and useful category, researchers use these alternative classes and methods to bridge the gap between abstract mathematical theory and the practical realities of computer science.

The complexity class FP

(Polynomial Time for Functions) represents the functional version of the class **P**. While P deals with decision problems (tasks with "yes/no" answers), **FP** is used to classify tasks where the goal is to produce a specific **output value**.

Here is a detailed breakdown of the components you provided, along with a concrete example from the sources:

Functions vs. Languages

In previous sections, we focused on **languages** (sets of strings), which correspond to decision problems. However, most practical tasks involve turning an input into a specific output (e.g., "What is the sum of these numbers?" rather than "Is the sum of these numbers 10?"). To handle these, we use a Turing Machine (TM) equipped with a dedicated **output tape**. A function f is "computable" if a TM can take an input x and leave the correct result $f(x)$ on that output tape.

$$FP = \bigcup_{c \geq 1} FDTIME(n^c).$$

These three statements describe the relationship between decision problems (languages in **P**) and functional problems (functions in **FP**). Here is a detailed explanation of each part with examples.

$L \in P \rightarrow \text{Characteristic Function } f \in FP$

"For every $L \in P$, the characteristic function f of L is trivially in **FP**."

A **characteristic function** is a boolean function that represents a decision problem. For any language L , its characteristic function f works as follows:

- It outputs **1** if the input string x belongs to the language L .
- It outputs **0** if the input string x does not belong to the language L .

If a problem is in **P**, it means there is a Turing Machine (TM) that can decide whether $x \in L$ in

polynomial time. Since this same TM can be configured to write the answer (0 or 1) on an output tape, it is technically computing the function f in polynomial time. Therefore, every decision problem in P has a corresponding function in the class **FP**.

- **Example:** Consider the language of **Palindromes** (L). Because there is an algorithm that can decide if a string is a palindrome in $O(n)$ time, the function $f(x)$ —which simply returns 1 for "yes" and 0 for "no"—is a member of **FP**.

Turning Functions into Languages

"For certain classes of functions (e.g. those corresponding to optimization problems), there are canonical ways to turn a function f into a language L_f ."

In many cases, we want to convert a task that requires finding an answer (a function) into a task that only requires a "yes/no" answer (a language). This is common for **optimization problems**, where the goal is to find the "best" or "largest" solution.

The "**canonical way**" to do this is to introduce a threshold, usually a natural number k , and ask if a solution exists that is at least as good as k .

- **Example: The Wedding Problem.**
 - **The Function (f):** "Given a list of guests and their incompatibilities, find the size of the **largest possible group** of compatible guests".
 - **The Language (L_f):** "Given a list of guests and a number k , is there a group of compatible guests of **size at least k** ?". This allows researchers to study the difficulty of the underlying problem using the tools of decision complexity (like NP-completeness).

Why $f \in FP$ Does Not Imply $L_f \in P$ (General Case)

"In general, however, it is not true that $f \in FP$ implies $L_f \in P$."

While every problem in P can be viewed as a function in FP, the reverse is not always true because **FP is a much broader class**. Functions in FP can produce outputs of any length (as long as the length is polynomial), whereas problems in P are restricted to a single bit of output (0 or 1).

This statement serves as a reminder that being able to compute a specific value efficiently does not mean that every possible "yes/no" question you could ask about that value is also efficiently decidable.

- **Example:** Imagine a function f in **FP** that performs a very complex transformation on a string. While we can compute the result $f(x)$ quickly, we might not be able to decide complex properties about that result (the language L_f) in polynomial time if those properties require extra information or a different type of processing that doesn't fit the standard "decision" model.

In summary, **P is essentially a small subset of FP** consisting only of functions that output a single bit. Because FP contains much more complex tasks, the two classes are not perfectly interchangeable.

Example Problems in **P** or **FP**

- ▶ **Lists:** inverting a list, sorting a list, finding the maximum or minimum element in a list, etc.
- ▶ **Graphs:** reachability, shortest paths, minimum spanning trees, etc.
- ▶ **Numbers:** primality test, exponentiation, etc.
- ▶ **Strings:** string matching, approximate string matching, etc.
- ▶ **Optimization Problems:** linear programming, maximum cost flow, etc.

How to Prove a Task Being in **P** or **FP**

We use some pseudo code and we calculate the complexity of the algorithm

- ▶ **Example.**
 - ▶ Suppose you want to show the following problem to be computable in polynomial time: given two strings $x, y \in \{0, 1\}^*$. determine if x contains an instance of y .
 - ▶ A pseudocode solving the problem above is the following:

```
 $i \leftarrow 1;$ 
while  $i \leq |x| - |y| + 1$  do
  if  $x[i : i + |y| - 1] = y$  then
    return True
  else
     $i \leftarrow i + 1$ 
  end
end
return False
```


- ▶ The input can be easily encoded as a binary string.
 - ▶ The total number of instructions is polynomially bounded.
 - ▶ Indeed it is $O(|x|)$.
 - ▶ All intermediate results are polynomially bounded in length.
 - ▶ Indeed, i cannot be greater than $O(|x|)$, thus its length is $O(\lg |x|)$.
 - ▶ Each instruction takes polynomial time to be simulated.
 - ▶ Comparing two strings of length $|y|$ can be done in polynomial time in $|y|$, thus polynomial in $|\perp(x, y)\perp|$.
-

The Class **EXP**

- ▶ The *next* class of functions $T : \mathbb{N} \rightarrow \mathbb{N}$, beyond the polynomials and having nice closure properties is the class of exponential functions.
- ▶ The classes **EXP** and **FEXP** are defined as follows:

$$\mathbf{EXP} = \bigcup_{c \geq 1} \mathbf{DTIME}(2^{n^c}) \quad \mathbf{FEXP} = \bigcup_{c \geq 1} \mathbf{FDTIME}(2^{n^c})$$

- ▶ Of course, it holds that

$$\mathbf{P} \subseteq \mathbf{EXP} \quad \mathbf{FP} \subseteq \mathbf{FEXP}$$

EXERCISES

Exercise 1

Exercise 1 asks you to prove that the function **minmax** is in the complexity class **FP**.

The task for this function is as follows: given an encoded list of natural numbers $(s_1, \dots, s_m)$, it must return both the **minimum** and the **maximum** value found within that list. For example, $\text{minmax}(2,$

7, 0, 2, 1, 3) would return (0, 7).

Solution Explanation

To prove a function is in FP (Polynomial Time for Functions), one must show it can be computed by a Turing Machine in a number of steps that is polynomially bounded by the input size. The solution provided in the sources breaks this down into three main points based on an algorithmic pseudocode:

1. The Algorithmic Process

The solution describes a simple algorithm to find the values:

- **Initialize** two variables, min and max, with the first element of the list (s_1).
- **Iterate** through the rest of the list (from $i = 2$ to m).
- **Update** min if the current element s_i is smaller than the stored min.
- **Update** max if the current element s_i is larger than the stored max.
- **Return** the pair (min, max).

```
min ← s1
max ← s1
for i ← 2 to m do
  if si < min then
    min ← si
  if si > max then
    max ← si
return (min, max)
```

Handwritten annotations on the right side of the pseudocode:

- A red bracket groups the first two lines (`min ← s1` and `max ← s1`) with a red "2" next to it.
- A red bracket groups the loop body (`if si < min then`, `min ← si`, `if si > max then`, `max ← si`) with a red " $O(m)$ TIMES" next to it.
- A red bracket groups the two inner if-statements with a red "AT MOST 4 INSTRUCTIONS" next to it.
- A red bracket groups the final `return (min, max)` line with a red "1" next to it.

REMIER:

Definition of Data Size

In the context of a Turing Machine (the standard model for these problems), data size refers to the **number of cells** used on the machine's tapes (the "scratchpad") to store variables, intermediate calculations, and results.

- For a string s , the size is its length $|s|$.
- For a natural number m , the size is the number of bits required to represent it in binary, which is approximately $\log_2(m)$.

2. Complexity Analysis

The proof that this algorithm places the task in FP relies on the following analysis:

• Total Number of Instructions:

The algorithm exhaustively looks at each of the m elements exactly once. The loop executes $O(m)$ times, and because each iteration contains a fixed number of elementary instructions (at most 4), the total number of instructions is $O(n)$, where n is the total length of the input.

- **Size of Intermediate Results:**

In complexity theory, particularly for the class **FP** (Polynomial Time for Functions), the requirement for **Polynomial Data Size** ensures that the "space" or "bit-length" of every piece of information processed by an algorithm remains manageable.

Here is a detailed explanation of this concept based on the sources:

The Polynomial Constraint

For a problem to be in FP, all intermediate results generated during the computation must be **polynomially bounded in length** relative to the input size n . This means if the input has n bits, any variable v created by the algorithm must satisfy: $\text{bit-length}(v) \leq p(n)$ where p is some polynomial function (like n^2 or $10n$).

this exercise

- **Variables Derived from Input:** In the **minmax** exercise, the variables **min** and **max** are simply copies of elements already present in the input list. Because they are taken directly from the input string of length n , their size is naturally $\leq n$, which is a polynomial.
- **Counters and Indices:** In search algorithms (like finding a substring), we often use an index variable i to track our position.
 - If i tracks a position in a string of length n , then $i \leq n$.
 - The bit-length of i is therefore $O(\log n)$.
 - Since $\log n$ is much smaller than n , this is well within the polynomial limit.

Summary: Polynomial Data Size is a safeguard. It ensures that an algorithm doesn't "cheat" by creating massive numbers or strings that would eventually require an exponential amount of time to process or even just to store on the machine's tape.

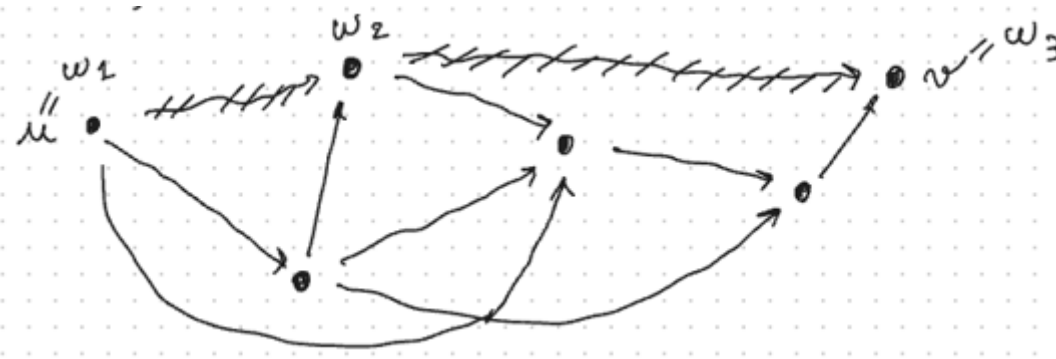
- **Execution Time per Instruction:**

Each individual instruction (like comparing two natural numbers) can itself be executed in polynomial time relative to the number of bits in those numbers.

Conclusion: Because the algorithm uses a polynomial number of instructions, keeps intermediate data to a polynomial size, and uses instructions that are themselves efficient, the minmax function is confirmed to be in **FP**.

Exercise 2

The task is to determine, given a directed graph $G = (V, E)$ and two specific nodes $u, v \in V$, whether v is reachable from u . Reachability means there exists a sequence of nodes $w_1, \dots, w_n$ such that $w_1 = u$, $w_n = v$, and every consecutive pair (w_i, w_{i+1}) is an edge in the graph.



The Proposed Algorithm

To prove the problem is in **P**, the source provides a search algorithm (similar to Breadth-First Search) described in pseudocode:

1. **Initialize:** Create a set called visited containing only the starting node u , and a list called current (the frontier) also containing u .
2. **Explore:** While the current list is not empty:
 - Pick and remove a node w from current.
 - For every neighbor t of w (where (w, t) is an edge):
 - If t has not been visited yet, add it to both the visited set and the current list.
3. **Result:** If v was ever added to the visited set, return **true**; otherwise, return **false**.

```

visited ← [u]
current ← [u]
while current ≠ [] do:
    pick w from current
    remove w from current
    for each (w,t) ∈ E
        if t ∉ visited
            add t to visited
            add t to current
    if v ∈ visited then
        return true
    else
        return false
  
```

2
] $O(n)$ TIMES
 2
] $O(n)$ TIMES
 3
 4

Complexity Analysis (Why it is in P)

The algorithm is shown to be efficient based on three main criteria:

- **Total Iterations:** The while loop executes at most $|V|$ times (the number of vertices). This is because the visited set only grows ("inflationary") and nodes are only added to current if they haven't been visited before, ensuring no node is processed more than once.
 - **the for each:** The for each that check all the nodes connected to *Current* will be executed $|V|$ because each node can be connected to all the other nodes
 - **Instruction Complexity:** All the other individual step within the loop involves basic list manipulations (adding/removing elements or checking membership) or checking. So the complexity of these operations is 1
- for those reason the number of macro-instructions is $O(n^2)$

- **Space Requirements:** The variables visited and current only store vertices from the graph without duplicates, meaning their size is always polynomially bounded by the number of vertices n .

Proof of Correctness of the Algorithm

A formal proof by **induction on the path length n** :

- **Base Case ($n = 0$):** If $v = u$, it is inserted into visited at the very beginning.
- **Inductive Step:** If v is reachable via a path of length k , there must be a predecessor w such that $u \rightarrow \dots \rightarrow w \rightarrow v$. By the inductive hypothesis, w will eventually be visited and added to current. Once w is processed, the algorithm will check the edge (w, v) and successfully add v to the visited set.

Exercise 3

The exercise focuses on proving that the **Wedding Problem** (or Wedding Plan Problem) belongs to the complexity class **FEXP** (Exponential Time for Functions).

Problem Definition

The sources define the Wedding Problem as follows: given a set L of candidate invitees and a list I of incompatible pairs, the goal is to find the **largest possible subset** of L consisting only of compatible guests.

In mathematical terms, this is modeled as the **Maximum Independent Set** problem on an undirected graph $G = (V, E)$, where:

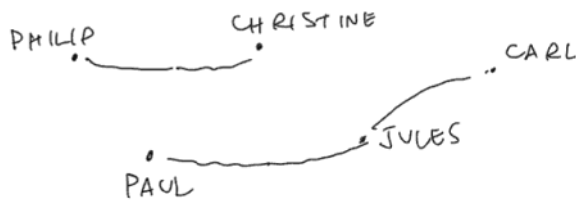
- **V (Vertices):** The set of candidate invitees.
- **E (Edges):** The set of incompatibility constraints between them.
- **Independent Set W :** A subset of nodes where no two nodes are connected by an edge (i.e., no two invitees in the set are incompatible).

• THIS CAN BE SEEN AS A PROBLEM ON GRAPHS, E.G.



$$G = (\{ \text{PHILIP, CHRISTINE, ...} \}, \{ \{ \text{PHILIP, CHRISTINE} \}, \{ \text{PAUL, JULES} \}, \{ \text{JULES, CARL} \} \})$$

• THIS CAN BE SEEN AS A PROBLEM ON GRAPHS, E.G.



$$G = (\{PHILIP, CHRISTINE, \dots\}, \{\{PHILIP, CHRISTINE\}, \{PAUL, JULES\}, \{JULES, CARL\}\})$$

The Proposed Exercise Solution

The exercise provides an algorithmic approach to solve this by **exhaustive enumeration**:

1. **Iterate through all subsets:** The algorithm considers every possible subset W of the set of invitees V .
2. **Check for independence:** For each subset, it verifies if it is "independent" by checking if any node w in the subset has a neighbor (an incompatible person) also present in the subset ($N(w) \cap W \neq \emptyset$).
3. **Track the maximum:** It keeps track of the largest independent set found during this process and returns it at the end.

$Z \leftarrow \emptyset$	] 1	
foreach $W \subseteq V$ do	] $O(2^{ V })$	
$ind \leftarrow \text{True}$	] 1	$2 + O(2^{ V }) \cdot (4 + O(V) \cdot 2)$
foreach $w \in W$ do	] $O(V)$	$= O(2^{ V }) \cdot O(V)$
if $N(w) \cap W \neq \emptyset$ then	] ≤ 2	$= O(V \cdot 2^{ V })$
$ind \leftarrow \text{false}$	] ≤ 3	$= O(2^{ V ^2})$
if ind then		
if $ Z < W $ then		
$Z \leftarrow W$		
return Z	] 1	

REMINDER:

What does the "size of G (graph)" ($|G|$) mean?

In the context of these sources, $|G|$ (or $\lfloor G \rfloor$) refers to the **length of the string used to encode the graph**, which depends on the chosen representation.

- If the graph is represented by an **adjacency matrix**, $|G|$ is approximately $|V|^2$.
- If the graph is represented by **adjacency lists**, $|G|$ is proportional to $|V| + |E|$. Therefore, the "size of G " encompasses both the number of nodes (Vertices) and the connections (Edges) between them, representing the total input size for the graph.

Here is the step-by-step breakdown of the algorithm and the computational cost associated with each phase:

1. Initialization

- **Action:** A variable Z is created and initialized as an empty set. This variable is used to store

the **largest compatible subset** found during the execution.

- **Cost: $O(1)$.** This is a simple assignment operation.

2. Main Loop (Subset Generation)

- **Action:** The algorithm iterates through **every possible subset W** of the total set of invitees V . So the line *foreach $W \subseteq V$* is used to summarize the subset W generation
- **Cost: $O(2^{|V|})$.** For each invitee, there are two choices: they are either in the subset or they are not. This results in 2^n total combinations (where n is the number of invitees). This step is what makes the algorithm **exponential** and impractical for large inputs.

3. Inside the Loop

For every generated subset W , the algorithm must verify if all members are compatible:

- **Action:** It sets a boolean flag *ind* to True. Then, for each person w in the subset W , it checks if any of their "neighbors" (the people they are incompatible with, denoted as $N(w)$) are also present in W .
- **Cost: Polynomial,**
The initial boolean setup is $O(1)$.

- Second loop cost

While running through all the elements of W is approximately $O(|V|)$. Because the biggest possible size of W is when all the nodes are part of the group. While for the Neighborhood check, if we consider G as an **adjacency matrix**, getting the current w column of the matrix and performing a bitwise operation with Z takes 2 basic operations. So $N(w) \cap W \neq \emptyset$ is just $O(2)$.

the total cost of this step is $O(1 + |V| * 2)$

4. Check of Z

The check to update Z are maximum 3 basic operations, so the total cost of this step is $O(3)$

5. Final Output

- **Action:** After checking all $2^{|V|}$ subsets, the algorithm returns the final set Z .
- **Cost: $O(1)$.**

Summary of Total Complexity

The overall computational cost is

$$1 + 2^{|V|} * (1 + |V| * 2 + 4)$$

$$= 1 + 2^{|V|} * (5 + |V| * 2)$$

$$= O(2^{|V|} * |V|)$$

And because $2^{|V|}$ is always smaller than $|V|$ for large V , the professor has chosen to write:

$$O(2^{|V|^2})$$

Space: All intermediate results (like the current largest subset) are polynomially bounded in size, which is a requirement for these complexity classes.

Because the running time is bounded by an exponential function of the input size, the problem is classified as belonging to the complexity class **FEXP** (Exponential Time for Functions).

Prof That: $P \subsetneq EXP$

The theorem states that the complexity class **P** is a **strict subset of the class EXP**, formally written as $P \subsetneq EXP$.

Here is a detailed explanation of the theorem and its proof as presented in the sources:

Meaning of the Theorem

- **Inclusion ($P \subseteq EXP$):** This part is considered obvious because any problem solvable in polynomial time (n^c) can also be solved if given exponential time (2^{n^c}).
- **Strict Inclusion ($\subsetneq$):** The core of the theorem is that there are problems in EXP that are **intrinsically intractable**, meaning they cannot be solved in polynomial time regardless of the algorithm used.

Proof Sketch (Hierarchy Theorem)

The proof relies on the **Time Hierarchy Theorem**, which mathematically demonstrates that providing a machine with "much more time" allows it to solve more problems.

The logic follows these steps:

1. **The Hierarchy Rule:** A simplified version of the Time Hierarchy Theorem states that for every "reasonable" function f , the class of problems solvable in time $f(n)$ is a strict subset of those solvable in $f^2(n)$. This is written as $DTIME(f(n)) \subsetneq DTIME(f^2(n))$.
2. **Building the Chain:** We can apply this rule to create a sequence of inclusions:
 - P is contained within $DTIME(2^n)$.
 - According to the hierarchy theorem, $DTIME(2^n)$ is a **strict subset** of $DTIME((2^n)^2)$, which is $DTIME(2^{2n})$.
 - $DTIME(2^{2n})$ is itself contained within the general definition of EXP .
3. **Conclusion:** Because there is a "gap" (a strict inclusion) in the middle of this chain ($DTIME(2^n) \subsetneq DTIME(2^{2n})$), it follows that the starting class (P) must be different from the ending class (EXP).

In essence, this theorem provides the mathematical certainty that **not all computable tasks can be solved efficiently**; some problems are fundamentally too hard for polynomial-time algorithms.